SMART INDIA HACKATHON 2026 · INTERNAL ROUND

PHISHEYE

Analyze before you trust. A working SIH1454 prototype.

PROBLEM ID
SIH1454

ORGANISATION
NTRO

THEME
Blockchain & Cybersecurity

CATEGORY
Software

TEAM ID
[fill from SIH portal]

TEAM NAME
[fill from SIH portal]

Official title: Create an intelligent system using AI/ML to detect phishing domains which imitate look and feel of genuine domains.

DETECT	UNDERSTAND	EXPLAIN	PROTECT	LEARN

Honesty strip — say this in the first two minutes.
FastAPI, not Flask. SQLite file *threats.db*, not live PostgreSQL. No trained CNN, BERT, or TensorFlow in this repository. Theme includes blockchain; we did not build a ledger. PhishShield is the SIH idea title. The running product is **PHISHEYE**.

Closing sentence (memorise this):
PHISHEYE is a working SIH1454 prototype: one FastAPI threat engine, Simple and Technical views, public threat feeds plus lookalike heuristics, and a Chrome gate that opens clear sites and keeps the rest closed — without claiming a trained CNN, live Postgres, or inbox access we do not have.

How to use this pack

Print this PDF. Give every speaker a copy. Fill Team ID and Team Name from the SIH portal on the cover by hand. Do not invent them.

1. **One-page cheat sheet** — keep this on the table during Q&A.
2. **60-second pitch** — if they cut you short.
3. **12-minute jury script** — timed spoken lines and demo clicks.
4. **3-minute emergency script** — Wi-Fi or laptop dies.
5. **Six SIH slide notes** — what to say on each official slide.
6. **Architecture diagrams** — draw these on the whiteboard if asked.
7. **40 viva answers** — 20–45 seconds, plus push-back lines.
8. **Faculty tactics** — how Indian college panels actually mark.

> **Must know, all four speakers:** Website and Chrome extension both call `POST /api/v1/threat/check-url`. Same URL, same score. Same FastAPI brain on port 8000.

> **Never say:** Flask · live PostgreSQL · trained CNN / 99% accuracy · we read Gmail or phone SMS · padlock means genuine SBI · community report is legal proof · we built a blockchain.

Source of truth: the current repository. If a fact is not in the code, say "not in our current build." Never invent a filename or an accuracy number.

G. One-page cheat sheet — print this first

Problem

SIH1454 · NTRO · Software

Detect phishing domains that imitate genuine sites (paypa1, sbi-login.xyz, amaz0n).

Theme is Blockchain & Cybersecurity. Our work is **cybersecurity**. We did not implement a ledger.

Product

PHISHEYE — website + Chrome extension + one FastAPI engine.

PhishShield AI = SIH idea title. CyberGuard = old code name. Say PHISHEYE to the jury.

Five verbs: Detect · Understand · Explain · Protect · Learn

Stack and ports

React 19 + Vite 8 · `http://127.0.0.1:5173`

FastAPI + Uvicorn · `http://127.0.0.1:8000` · Swagger `/docs` · health `/health`

SQLite `backend/threats.db` · JWT + bcrypt · OpenCV QR · MV3 1.8.0

Demo URLs (never open a live kit)

`https://www.google.com` → LOW, auto-open

`https://paypa1.com` or `https://sbi-login.xyz` → HIGH/CRITICAL, stay closed

SMS paste: "Urgent: Your SBI account will be blocked... Enter OTP here: https://sbi-login.xyz/verify"

Intelligence today

Feeds: OpenPhish, URLhaus, PhishTank dump, Phishing Army, optional Google Safe Browsing.

Lookalikes: fragments, leet (1→l), Levenshtein, limited homoglyphs, free-host + login tokens.

Psychology: English + Hinglish keyword rules. Cap 35 if it is the only signal.

The fusion key named `ml` is a **copy of the lexical URL score**. No trained classifier.

Honesty lines

Intel `no_match` is not SAFE. Timeout is UNAVAILABLE, not SAFE.

HTTPS / padlock does not mean genuine SBI.

Model confidence = how many signals agreed, not accuracy.

Community Mark Scam is a signal, not court proof. Rate limited: 1 URL / 24h, 20 / day.

QR is decoded. Destination is not visited. Images are not stored.

Close with this sentence, then stop talking for two seconds:
PHISHEYE is a working SIH1454 prototype: one FastAPI threat engine, Simple and Technical views, public threat feeds plus lookalike heuristics, and a Chrome gate that opens clear sites and keeps the rest closed — without claiming a trained CNN, live Postgres, or inbox access we do not have.

A. 60-second elevator pitch

Lead speaks. Stand beside the screen. Do not read the title slide word for word.

Good morning. Problem SIH1454 from NTRO asks for an intelligent system that detects phishing domains which imitate genuine sites — SBI, PayPal, IRCTC, UPI. In India, one WhatsApp KYC message is enough. The page has a green padlock. A parent types the OTP. Money leaves. HTTPS is encryption. It is not proof that the brand is real.
Our product is PHISHEYE. It is a working prototype, not a slide. Users get a website and a Chrome extension. Both call one FastAPI engine. We detect lookalikes, we explain the result in Simple English and Technical evidence, and the extension scans before the other site is used. Clear sites auto-open. Dangerous sites stay closed.
We did not train a CNN. We did not build a blockchain. The live database is SQLite. Intelligence today is explainable: public threat feeds, brand distance, Hinglish psychology rules, and a decision engine that will not let a confirmed feed hit be averaged away. Thank you — we can demo in the next minute.

C. 3-minute emergency script

Use this if Wi-Fi dies, uvicorn crashes, or the projector will not show the live app. Show desktop screenshots: ScanResult CRITICAL, gate closed, phone photo of Swagger `/docs`.

0:00–0:40	LEAD	SIH1454, NTRO: detect lookalike phishing domains. Product name PHISHEYE. Theme includes blockchain; we did cybersecurity, not a ledger. FastAPI, not Flask. SQLite file threats.db, not live Postgres. No trained CNN in this repository.
0:40–1:20	LEAD	Point at the CRITICAL screenshot. This is Simple View: do not enter OTP. paypa1.com is not paypal.com. Digit 1 instead of L. That is the problem statement — imitate look and feel.
1:20–2:00	EXT	Point at the gate screenshot. We scan before the destination is used. Google would auto-open. This lookalike stayed closed. Chrome may flash a blank page on a typed URL. Link clicks do not load the phish first.
2:00–2:35	BACK	Four honesty lines: miss is not safe. Psychology cannot solo-CRITICAL — we cap it at 35. We do not read Gmail or phone SMS. Confidence is signal agreement, not accuracy.
2:35–3:00	LEAD	Closing sentence. Sit down. Do not apologise for the laptop.

B. 12-minute jury script

Zoom browser 125%. Hide WhatsApp and Discord. Boot both servers in the corridor. Lead keeps time. Floater restarts uvicorn. Never debug silently — narrate every click.

> **Timer rule:** never cut the live demo to save architecture. Cut architecture instead. Two live URLs only: Google, then paypa1 or sbi-login.xyz.

Open — problem and product

0:00–0:45	LEAD	Good morning. Problem SIH1454 from NTRO: detect phishing domains that imitate genuine domains. Theme is Blockchain and Cybersecurity; our work is cybersecurity, not a blockchain. In India, a fake SBI or UPI link on WhatsApp is enough to steal an OTP even when the padlock is green. HTTPS does not mean the brand is real.
0:45–2:00	LEAD	Our product is PHISHEYE. It is a working prototype, not a slide. Five verbs: Detect, Understand, Explain, Protect, Learn. Users get a website and a Chrome extension. Both call one FastAPI engine. If a slide still says PhishShield, that is the SIH idea title. The running product is PHISHEYE.

Stack — Backend speaks

2:00–2:45	BACK	Stack: React 19 and Vite on port 5173. FastAPI and Uvicorn on port 8000. Open `/docs` if they doubt FastAPI. SQLite file threats.db — PostgreSQL is planned, not live. Auth is bcrypt and JWT. QR is OpenCV. Tests are pytest.
2:45–3:30	BACK	Intel is OpenPhish, URLhaus, PhishTank dump, Phishing Army, optional Google Safe Browsing. Pipeline: INPUT, NORMALIZE, ANALYZE, FUSE, EXPLAIN, ACT. Modular monolith. One Python process. Not microservices.

Intelligence — still Backend

3:30–4:20	BACK	We normalize the URL so www, default ports, and query strings cannot trick us. We query public threat feeds. A miss is no_match, not SAFE. A timeout is unavailable, not SAFE. If a feed confirms a URL, heuristics cannot override it — confirmed intel floors the score around 80 to 85 for an exact match.
4:20–5:10	BACK	Separately we catch lookalikes: paypa1, amaz0n, Levenshtein distance to SBI and PayPal, free hosts like vercel.app with login in the name. Psychology is English and Hinglish phrases — rules, not BERT. Psychology alone cannot go CRITICAL. We cap it at 35 so a marketing SMS is not a phishing alert.
5:10–5:30	BACK	There is no trained CNN in this repository. The channel named ML is a copy of the lexical URL score so nobody can say “ML said 10 therefore it is safe.” A supervised model is phase two after curated labels. Famous sites: Google, YouTube, GitHub are trusted so one dump row for a watch video does not burn all of YouTube. sites.google.com and github.io stay fully scanned.

Website — Frontend speaks

5:30–6:30 — FRONT

The site: land, sign up, dashboard, scanner with six tabs — URL, Message, Email, WhatsApp/SMS, Screenshot, QR. Last result has Simple View for parents: do not enter OTP. Technical View for analysts: providers, registered domain, tags. Then history, community reports, statistics from real SQLite rows, privacy, advisor FAQ — not ChatGPT — and settings.

Live demo — Extension narrates, Lead clicks

Lead does the mouse. Extension speaker talks. Backend stays quiet unless the API fails.

6:30–6:50 — FRONT

Landing page. This is PHISHEYE. Same engine as the Chrome extension.

> Click: open `http://127.0.0.1:5173`. Point at the PHISHEYE mark. Log in with the prepared account.

6:50–7:25 — FRONT

Scan, URL tab, official domain. No feed match is not a 100% guarantee, but heuristics are clean and this is an official domain.

> Click: Scan → URL → `https://www.google.com` → Last Result. Point at LOW, Simple View, Technical intel no_match.

7:25–8:00 — FRONT

Lookalike. Digit 1 instead of L. Brand impersonation. Simple View: do not enter OTP.

> Click: scan `https://paypa1.com` or `https://sbi-login.xyz`. Point at the score and the DANGER verdict. Do not open the site in a raw tab.

8:00–8:35 — FRONT

WhatsApp and SMS tab. Chrome cannot read phone SMS. The user pastes visible text. Psychology tags plus the link HIGH.

> Paste: *Urgent: Your SBI account will be blocked. Complete KYC immediately. Enter OTP here: https://sbi-login.xyz/verify*

8:35–9:00 — FRONT

QR tab. We decode the QR. We do not visit the site.

> Upload the prepared PNG. Point at `decoded_text` and the score.

9:00–9:30 — EXT

Extension gate. We scan before the other site is used. Clear sites auto-open. MEDIUM asks Open anyway. HIGH and CRITICAL stay closed. Scan every page is off so we do not re-scan every GitHub click. Chrome may flash a blank page on a typed URL. Link clicks do not load the phish first.

> Type `github.com` → gate → auto open. Then a lookalike → stays closed.

Honesty and close

9:30–10:20 — LEAD

We do not read Gmail or WhatsApp inboxes. We do not read phone SMS. We do not store uploaded images. We do store scan metadata in SQLite — be honest about that. Community Mark Scam is a signal, not a court proof. Model confidence is signal agreement, not accuracy. CORS is open for the prototype. We will not claim 99 percent.

10:20–11:00	FRONT	Statistics charts come from real scan rows in SQLite. Advisor is FAQ, not ChatGPT inside the app.
11:00–12:00	LEAD	Impact: fewer OTP thefts for students and families; a local engine campuses can run; a path to a trained model when NTRO-scale labelled data exists. Closing sentence. Thank you. We are ready for questions.

> Open Statistics, then Advisor. One sentence each. Do not tour CSS.

> After questions, if they look unconvinced, repeat the closing sentence once. Then stop.

D. Six official SIH slides — speaker notes

Fill Team ID and Team Name from the portal on slide 1 before you walk in. If a slide still says PhishShield: “PhishShield is the SIH idea title. The running product is PHISHEYE.”

Slide 1 — Title

Say the problem ID and the full title once, slowly. Say NTRO, Software, Blockchain and Cybersecurity. Then one sentence: we built cybersecurity, not a blockchain. Do not spend time on the template chrome.

Slide 2 — Idea

PHISHEYE: analyze before you trust. Multi-channel input — URL, pasted SMS, QR, screenshot, browser tab — one engine — Simple and Technical views. Innovation is five verbs: Detect, Understand, Explain, Protect, Learn. Read the honesty footer: no inbox OAuth, OCR only if Tesseract is installed, CNN planned, confidence is not accuracy.

Slide 3 — Technical approach

React, Vite, Tailwind, Axios, Recharts. FastAPI, Pydantic, Uvicorn. SQLite live. URLChecker, Psychology 2.0, decision engine, fusion, OpenCV QR, Manifest V3, bcrypt, JWT, SSRF. Architecture line: INPUT → NORMALIZE → ANALYZE → FUSE → EXPLAIN → ACT. Not Flask. Not live Postgres. Not a TensorFlow CNN.

Slide 4 — Feasibility

Technical: runs on a campus laptop with Python and Node. Operational: threat feeds cache 30 minutes on disk. Security: SSRF guard, bcrypt, report caps, 5 MB upload limit. Challenges we already mitigated: psychology false positives capped at 35; famous-site dump noise handled; intel miss is not treated as safe.

Slide 5 — Impact

Users: citizens and parents on Simple View; students and campus analysts on Technical View. Before: padlock plus a panic SMS. After: scan first, plain English, do not enter OTP. KPIs on the slide are proposed — time-to-warn, false-open rate — not claimed production results. We have no production user count.

Slide 6 — Research

SIH1454 / NTRO, Google Safe Browsing docs, OpenPhish, URLhaus, PhishTank, RDAP, OWASP phishing and SSRF, FastAPI, React, Chrome MV3, SQLite, OpenCV. No fake IEEE papers. If they ask for a citation we do not have: “That paper is not in our references. Our implementation is in these files.”

E. Diagrams you can redraw on a whiteboard

1. System architecture — one brain, many mouths

```
[ React dashboard :5173 ]     [ Chrome extension gate / popup ]
                  \                      /
                   \     HTTP JSON      /
                    v                   v
                  [ FastAPI :8000  —  PHISHEYE API v2.0.0 ]
                     |
                     +-- URLChecker (single URL)
                     +-- analyze_content (message / email / WA / QR / screenshot)
                     +-- threat_intel (feeds; never fetches the attacker page to decide intel)
                     +-- SQLite  backend/threats.db
                     v
                  [ Simple View + Technical View JSON ]
```

INPUT	NORMALIZE	ANALYZE	FUSE	EXPLAIN	ACT
URL, text, QR, tab	host, path, query	intel, brand, psych	0–100 + floors	Simple + Technical	save / open / refuse

2. Website page map

Path	Who	What to say
`/` Landing	Public	Brand PHISHEYE. Guest can scan URL or screenshot.
`/login` `/signup`	Public	JWT in localStorage key cg_token. bcrypt in SQLite.
`/dashboard`	01	Cards from real url_checks, not fake numbers.
`/scanner`	02	Six tabs: URL, Message, Email, WhatsApp/SMS, Screenshot, QR.
`/scan-result`	03	Score, Simple / Technical, intel, flag actions.
`/history` `/reports` `/statistics`	04–06	Past scans, community flags, Recharts.
`/privacy` `/advisor` `/settings`	07–09	Honest privacy. Advisor is FAQ, not an LLM.

3. URL scan sequence

A Normalize	B Intel	C Network	D Heuristics	E Decision	F Explain	G Persist
drop #fragment	feeds, never fetch page	DNS, age, TLS, SSRF	lookalike, TLD, IP	intel floors	two views	url_checks

Situation	Effect
Confirmed intel, exact URL	Floor score 85
Confirmed, normalized URL	Floor 80
Confirmed, hostname	Floor 72
Confirmed, registered domain	Floor 55
Reported, unverified	Floor 58, SUSPICIOUS
Feed miss + weak heuristics	NO_MALICIOUS_EVIDENCE — not a clean bill of health

4. Message / QR pipeline

normalize_content → psychology on full text → URLChecker on each extracted link → brand impersonation (mention + bad domain + extra scam signal) → optional sender From / Reply-To → community reputation → fuse() → if any child URL was confirmed malicious, fused score cannot fall below that URL's decision score → save content_scans if logged in.

QR: OpenCV decode, PIL fallback, 5 MB cap. Destination URL is not fetched. Screenshot: QR always; OCR only if Tesseract is installed on that PC.

5. Extension gate sequence

Click	Hold	Panel	Scan	LOW	MED	HIGH
gate-clicks.js	preventDefault	panel.html	check-url	auto-open	Open anyway?	stay closed

Session allow-list is by registered domain. Scan every page default OFF. Backend down → UNAVAILABLE, never SAFE. Typed omnibox URLs: Chrome MV3 cannot always cancel the first byte. Do not claim we always prevent it.

6. Decision engine vs fusion

URL path — decision engine

Used by URLChecker. Confirmed intel is a floor. Heuristics cannot wash a feed hit down to LOW. `ml_score` is the lexical/heuristic URL score copy.

Message path — weighted fuse

url 0.22, domain 0.16, ml 0.10, nlp 0.10, psychological 0.10, threat_intel 0.10, brand 0.08, sender 0.05, community 0.05, redirect 0.04. Starting defaults, not proven-optimal. Then raise the floor if a child URL was confirmed malicious.

Levels: LOW 0–29 · MEDIUM 30–49 · HIGH 50–69 · CRITICAL 70–100. `scam_risk` is the same integer, not a calibrated probability. `model_confidence` = 35 + 10 × (signals ≥ 20), cap 95.

F. 40 viva questions — spoken answers

Repeat the question in five words, then answer. One person answers. If you do not know: “That is not in our current build. Today we do X. It is on the roadmap.”

Q1. What is PHISHEYE?

Say: A multi-channel phishing and scam intelligence prototype: a React website, a Chrome Manifest V3 extension, and one FastAPI brain that scores URLs, pasted messages, QR codes, and the browser tab in Simple English and Technical evidence.

If they push back: If they say it is just a VirusTotal clone: we add lookalike heuristics, Hinglish psychology, an explainable decision engine, and a scan-before-open gate. We do not claim we replace national intel.

Q2. What is SIH1454?

Say: NTRO asked for an intelligent system using AI/ML to detect phishing domains that imitate the look and feel of genuine domains. Software track, Blockchain and Cybersecurity theme. We implemented cybersecurity.

If they push back: If they read the word AI/ML: intelligence today is multi-signal fusion plus threat intel plus lookalike heuristics. A supervised model is phase two after curated labelled reports.

Q3. Where is the AI?

Say: Explainable fusion, public threat feeds, and lookalike heuristics. There is no neural net in this repository. The fusion channel named ml is a copy of the lexical URL score so a low ML number cannot be used as proof of safety.

If they push back: If they demand a CNN file: there is no SavedModel, no sklearn pickle, no BERT. Show url_checker.py, decision_engine.py, risk_engine.py.

Q4. Why FastAPI, not Flask?

Say: The live API is FastAPI with Uvicorn. Pydantic validates bodies. /docs is automatic Swagger for the jury. Flask is not the running backend.

If they push back: If an old comment says Flask: leftover naming. Open http://127.0.0.1:8000/docs.

Q5. Is the database PostgreSQL?

Say: Planned for multi-user production. Live today is SQLite file backend/threats.db. SQLAlchemy model files exist but live queries use sqlite3 with parameterized placeholders.

If they push back: If they call that amateur: zero-ops MVP on a campus laptop. Scale path is Postgres. Do not say we run Postgres today.

Q6. How do you detect paypa1.com?

Say: Known phishing fragment list, leet table where digit 1 maps toward L, and Levenshtein distance to paypal. Combined with TLD and brand rules. That is lookalike detection, which is the heart of SIH1454.

If they push back: If they type paypal.com: official domain, trusted path, should be LOW if DNS exists.

Q7. Do you handle homoglyphs?

Say: Limited Cyrillic-to-Latin mapping inside the URL checker. Not a full Unicode IDN suite. We say limited on purpose.

If they push back: If they push punycode: we normalize the URL and score the host. Full IDNA hardening is not specified as complete in this build.

Q8. Why not blacklist only?

Say: Feeds lag. A new sbi-login.xyz can steal OTPs for hours before OpenPhish lists it. We add heuristics, brand distance, and psychology on the message text.

If they push back: If they say blacklists are enough: absence from a feed is no_match, not SAFE.

Q9. Why not ML only?

Say: We do not have a curated labelled production dataset yet. A black box fails viva. Parents need a sentence they can act on, not a silent probability.

If they push back: If they want F1: we have no official product F1. Closed-world scripts must be labelled simulations if mentioned at all. Prefer not to quote a percentage.

Q10. Is no_match safe?

Say: No. It means the feed was queried and nothing was found. Zero-day phishing will miss. We still run heuristics and brand checks.

If they push back: If they say then why use feeds: confirmed hits become floors so they cannot be averaged away.

Q11. What does unavailable mean?

Say: A feed timed out or errored. It is not no_match and it is not SAFE. The extension treats backend-down the same way: UNAVAILABLE, never SAFE.

If they push back: If they ask what the user sees: we do not auto-open. Dangerous default is keep closed.

Q12. Can psychology mark CRITICAL alone?

Say: No. If psychology is high and URL, domain, intel, brand, and community are all weak, fused score is capped at 35, which is LOW or MEDIUM. A sale SMS that says act now is not phishing.

If they push back: If they paste a bank OTP SMS with sbi-login.xyz: that is not psychology alone. The lookalike URL carries it.

Q13. Are community reports proof?

Say: No. Signal only, weight 0.05 in fusion. Logged-in users, rate limited: one URL per 24 hours per user, twenty reports per day. Not a court document.

If they push back: If they ask about brigading: that is why the caps exist. Still a prototype control, not a full abuse desk.

Q14. Does HTTPS mean the site is genuine?

Say: No. A padlock is encryption. Certificates are cheap. Fake SBI pages routinely have HTTPS. We even add a little risk if HTTPS is missing, but presence of HTTPS does not make it safe.

If they push back: If they show a green lock on a phish kit: that is the India story. OTP still leaves.

Q15. Do you visit the phishing page?

Say: Not to decide threat intel. Ads and clutter fetch is skipped if intel already confirmed, so we do not load the kit. QR does not visit. Shortener unwrap uses an SSRF guard.

If they push back: If they want screenshots of live kits: we refuse in the jury room. Use lookalikes and screenshots already saved.

Q16. What is SSRF protection?

Say: When we fetch an attacker-controlled URL, we block localhost, private RFC1918, and cloud metadata 169.254.169.254. That stops the scanner from becoming an open proxy into the campus network.

If they push back: If they ask for the file: backend/app/services/ssrf.py.

Q17. Does the extension read WhatsApp?

Say: Only visible links on WhatsApp Web after the page rendered. No native WhatsApp. No phone SMS. Chrome cannot read the phone.

If they push back: If they want Android SMS: planned companion, not implemented. mobile_source is not a live ingest.

Q18. Do you OAuth Gmail?

Say: No. Paste the email text, or type From and Reply-To, or scan visible links. Sender analysis only works if the user types those fields.

If they push back: If they say that is weak: it is an honest browser limit plus privacy. Inbox OAuth is roadmap if legally allowed.

Q19. Is QR a malware scanner?

Say: We decode the payload with OpenCV. We do not execute it. Then we score extracted URLs with the same engine. Destination is not fetched.

If they push back: If decode fails: say so. Try a clearer PNG, or paste the URL.

Q20. What about OCR?

Say: Optional Tesseract. If it is not installed on that PC, paste the text or use the QR tab. Partial, not a claim of full screenshot reading.

If they push back: Do not say we always read the screenshot.

Q21. How do you avoid burning YouTube?

Say: Official Google, YouTube, GitHub are trusted destinations. A dump row for one /watch video does not match every video. Query must overlap on generic paths. User-content hosts — sites.google.com, github.io — stay fully scanned.

If they push back: If they type youtube.com: expect LOW unless confirmed intel wins, which is rare and we still allow confirmed intel to win.

Q22. Are fusion weights optimal?

Say: No. They are documented starting defaults that sum to 1.0. We can change DEFAULT_WEIGHTS. We do not claim a grid-search paper.

If they push back: If they want proof: show risk_engine.py and say starting defaults.

Q23. SQL injection?

Say: Parameterized queries with question-mark placeholders. We do not concatenate user URLs into SQL strings.

If they push back: If they ask ORM: SQLAlchemy is in requirements and unused for live queries. Do not pretend we run an ORM today.

Q24. How are passwords stored?

Say: bcrypt via passlib. JWT HS256 after login. Token in localStorage key cg_token. Prototype, not a bank-grade KMS.

If they push back: If they want refresh tokens and rotation: not specified as implemented.

Q25. CORS?

Say: allow_origins star. Prototype limitation. Say it before they find it. Production must lock origins.

If they push back: Do not say we are production hardened.

Q26. Where is the blockchain?

Say: Theme includes cybersecurity. Phishing intelligence does not require a ledger for this MVP. We did not implement smart contracts.

If they push back: One sentence, then stop. Do not lecture on Ethereum.

Q27. What accuracy do you claim?

Say: None as a product guarantee. No official F1. Model confidence is agreement, not accuracy. We will not say 99 percent.

If they push back: If a teammate quoted a simulation: label it closed-world, stubbed network, not production.

Q28. Who is the user?

Say: Two people. A parent or first-time UPI user sees Simple View: do not enter OTP. A CSE student or campus analyst sees Technical View: providers, registered domain, tags.

If they push back: Do not say only CISOs. Do not say we arrest gangs.

Q29. What is future work?

Say: PostgreSQL, a trained model on curated reports, Android SMS companion, inbox OAuth if legally allowed, global API rate limit, CORS lockdown.

If they push back: Do not sell future work as built.

Q30. Why SQLite?

Say: Zero-ops MVP, one file, campus demo, init_db on API start. Charts read real rows. Scale later with Postgres.

If they push back: If the dashboard is empty: the API was not started, so the file was not initialized.

Q31. Why Manifest V3?

Say: Chrome requirement. Service worker. We cannot use MV2 blocking webRequest the old way, so we use a gate page: scan first, then open or refuse.

If they push back: Admit the omnibox blank-flash limit in the same sentence.

Q32. Is Advisor a chatbot?

Say: No. Static FAQ. There is no LLM classifier and no ChatGPT inside the app.

If they push back: If they used ChatGPT to write slides: tools helped; we own the design and the honesty.

Q33. Do you store images?

Say: No. Processed in memory. We do store URLs and scan metadata in url_checks and content_scans for history and charts.

If they push back: Do not say we store nothing.

Q34. How does Learn work?

Say: Logged-in Mark Scam, Risky, or Safe increments domain_reputation counters. Small fusion weight. Signal, not proof.

If they push back: If they mark Google as scam: rate limits plus trusted-domain logic. Still a prototype.

Q35. Decision engine versus fusion?

Say: URL path uses decision_engine so intel cannot be averaged away. Message path uses weighted fuse, then raises a floor if a child URL was confirmed malicious.

If they push back: If they want one formula: two pipelines, one product. That is intentional.

Q36. Free-host phishing?

Say: login, otp, bank tokens on vercel.app, lovable.app, pages.dev and similar extra-score. hdfc-login.vercel.app is a teaching example.

If they push back: Do not claim we vendor-block every PaaS.

Q37. Piracy sites?

Say: A separate name list and malvertising tags. We do not scan every torrent. It is extra signal, not a piracy product.

If they push back: Keep it one sentence. Return to lookalike banks.

Q38. Why NTRO?

Say: National-scale phishing harm on UPI, net banking, IRCTC, DigiLocker. They can audit an explainable prototype: feeds plus lookalikes match the problem statement.

If they push back: Do not claim official NTRO deployment.

Q39. What does the extension not do?

Say: It does not read passwords, OTPs, cookies, or SMS. It sends the URL, and optionally visible link URLs, to the local FastAPI engine.

If they push back: If they fear spyware: show the manifest permissions and say it out loud.

Q40. Show me a file.

Say: main.py FastAPI. url_checker.py lookalikes. decision_engine.py floors. risk_engine.py weights. psychology.py Hinglish. qr_analyzer.py OpenCV. extension/manifest.json version 1.8.0. connection.py SQLite schema.

If they push back: Never invent a filename. If you forgot, say so and open the repo map.

H. How Indian college panels actually mark

Typical internal: 8–12 minutes plus 3–5 questions. Panel is HOD, two CSE faculty, sometimes MBA or ECE, rarely a real NTRO person. They mix problem understanding, originality versus GitHub clones, technical depth, feasibility on this laptop, teamwork, and Q&A honesty.

To get shortlisted

1. Demo must work in the first minutes of the demo block. Boot servers in the corridor.
2. One story. If Lead says FastAPI and a teammate says Flask, you fail.
3. Map every feature back to imitate genuine domains. The extension is Protect. Do not let it steal the problem statement.
4. Dual audience: HOD hears Simple View and OTP. CSE hears decision floors and SSRF.
5. Admit limits in the same sentence as the mitigation. Faculty hate bluff more than a missing CNN.
6. Title card with Team ID, Team Name, PS ID. Internals are bureaucratic.
7. Two live URLs only: Google, then paypa1 or sbi-login.xyz.
8. Never cut the demo to save architecture.
9. Blockchain question: one sentence, stop.
10. Do not say ChatGPT built this.
11. Dark UI, 125% zoom. After feeds cache, airplane mode is allowed — TTL is 30 minutes on disk.
12. Print this PDF. Internal panels remember paper.
13. Practice with a friend whose only job is “show me the CNN.” Stay calm.
14. GitHub is a plus. Do not claim production users.
15. After questions, repeat the closing sentence if they look unconvinced.

Roles

Lead

Opens, closes, keeps time. Silent during demo clicks.

Backend

Engine, intel, SQLite. Silent on CSS.

Frontend

Pages, Simple / Technical. Silent on extension internals.

Extension

Gate, privacy, MV3 limit. Silent on fusion weights.

Floater restarts uvicorn and holds this PDF. Never argues with the HOD.

Green flags vs red flags

Green. Unavailable is not safe. Psychology cannot solo-CRITICAL. Same engine as Chrome. We can open /docs. pytest exists. Trusted-domain explanation when they type youtube.com.

Red. Arguing with HOD. Opening a live phish kit. Two people answering. Staring at Cursor. 99% ResNet. We use blockchain for URLs. Empty dashboard because the API was never started.

Body language

Stand beside the screen. Put the cursor on the score. Repeat the question in five words. Pause two seconds after the closing line. Simple formal dress. One laptop, one HDMI adapter.

Night before: cold start both servers, load the extension, run Google and paypa1, print this pack, sleep.

Forbidden claims

Flask · live PostgreSQL · trained CNN, ResNet, YOLO, BERT, TensorFlow SavedModel, sklearn pickle in this repo · intercept native WhatsApp or phone SMS · OAuth-read Gmail · 98–99% accuracy · padlock means genuine SBI · community report is legal proof · model confidence is accuracy · we store nothing · ZKP or federated learning implemented · blockchain implemented · we always block the first byte of a typed URL · we visit every phishing page to screenshot it · official NTRO deployment.

Closing sentence, last time:
PHISHEYE is a working SIH1454 prototype: one FastAPI threat engine, Simple and Technical views, public threat feeds plus lookalike heuristics, and a Chrome gate that opens clear sites and keeps the rest closed — without claiming a trained CNN, live Postgres, or inbox access we do not have.